多智能体协同开发系统搭建方案

多 Harness 协作优化版 2.0　2026 年 10 月 12 日

本版用于指导后续搭建。目标是让 Codex、DeepSeek Harness、Kimi Code、ZCode 四个真实 Harness 按职责协作，并优先复用开源实现。M0 将 CCTeam 作为跨 Harness 派发优先验证候选（公开支持 Codex、DSH、Kimi），Omnigent 作为隔离 worker/worktree 备选；Codex 负责规划与发起任务，外部状态机负责阶段审批。ZCode 适配是否可复用需单独验证，不能预设开箱即用。质量优先，不设降本目标。

1 本次调整与最终目标

四个 Harness 的真实接入是最终验收条件；切换模型或双 Agent 演示只算中间成果。

决策	本版处理
顶层结构	M0 对比 CCTeam 的跨 Harness 会话派发与 Omnigent 的隔离 worker/worktree；Codex 担任规划入口，外部状态机唯一审批阶段和派发。底座是否正式采用由 M0 结果决定。[1]
开发流程	六阶段保留为检查项，按任务合并执行；已批准范围内免逐工具审批，测试、独立评审和发布授权仍是硬门禁。
外部控制	保留独立于模型的确定性任务账本与权限控制，复用已有实现；不再新增第二套通用编排框架。
接入范围	四个 Harness 都纳入本期目标，分里程碑接通；DSH、ZCode 需要自定义适配，不能预报开箱即用。
模型与成本	Codex 使用官方 ChatGPT 订阅；其余沿用第三方云端 API。接受 DeepSeek 服务端路由，不自动切模，不做成本优化。

平台独立安装于 WSL2，以项目配置接入仓库。`D:\mingjing-develop\国庆后的原型` 作为真实试点。

运行关系为：用户 → Codex 规划会话 → 外部任务账本/权限网关审批 → 跨 Harness 派发层 → 指定 Harness 独立会话执行 → 确定性测试器 → 独立评审 → 用户授权交付。MCP/CLI 负责传递任务卡和产物引用，不把整个主会话拼成超长提示；原生工具与 MCP 工具均须受角色策略、网关和操作系统隔离约束。

四个 Harness 的初始职责

Harness	初始职责	权限与交付物
Codex	需求澄清、架构、任务拆分、选择已注册 Worker、终审	业务代码只读；通过受控接口管理任务。终审使用新的只读会话，读取 diff 与测试证据。
DeepSeek Harness	边界明确的功能实现、经批准的缺陷修复	独占任务 worktree；提交文件清单、diff 和说明。沿用策略 A，记录供应商返回的模型标识。
Kimi Code	大范围代码阅读、上下文整理、计划挑刺、独立审查	首期只读；输出文件定位、影响范围与问题清单。测试方案建议交给确定性测试器执行。
ZCode	前端页面、交互实现与 UI 问题修复	独占任务 worktree；提交实现和浏览器验收要求，共用受控浏览器验证。

职责优势须用同类任务验证；调整模型或新增写权限须显式记录。

2 开源复用与现有资产迁移

跨 Harness 派发优先检查 CCTeam：其常驻 daemon 用 MCP `agent/agent_read` 和 JS Flow 创建真实的 Codex、DSH、Kimi、OpenCode 等独立会话，并记录派发/回传；公开清单未列 ZCode。其 Flow 文档明确说明并行编辑共享 project working tree，未提供 per-hire worktree 隔离，因此需保留 ZCode 适配与写任务隔离工作。[14] Omnigent 列有 Codex（fully supported）和 Kimi（community-supported）；其 Polly 通过不同 Harness 的 worker 和独立 worktree 实现委派、交叉评审及各 worker 单独创建 PR。[2][3] Polly 默认 roster 不含 DSH、ZCode，但通用 ACP agent 和动态子会话配置可能提供接入路径，须在 M0 验证，不能把通用协议等同于现成兼容。

Polly headless worker 的文档示例使用 bypass 运行；虽说明 Omnigent 的 blast_radius policy 仍会拒绝灾难性操作，本项目仍须分别验证原生工具、MCP、OS sandbox 和审计覆盖，不能直接复制 bypass 配置。[3] 保留失败现场、人工确认后处置，不复用自动修复和自动回滚。Omnigent Alpha 版本固定后再运行本机验证。[1]

本次对照了三类高关注项目：AWS CLI Agent Orchestrator（CAO，约 1.4k stars）用本地 server 管理真实 CLI 与 tmux session，profile 显式绑定 provider，支持 Codex、Kimi、OpenCode；可参考 provider adapter、状态检测和 worker 生命周期，但不替代 SQLite 状态门禁。[10][11] Vibe Kanban（约 28.3k stars）以任务卡、agent attempt 和隔离工作区管理多种 CLI；适合参考任务隔离与人工看差异，官方文档没有证明其能在同一个任务中自动编排四个目标 Harness。[12][13]

Morning Star（约 63 stars）声明可在 Codex、DSH、Kimi Code、ZCode 等 Host 安装同一套 workflow/role plugin；每次由当前 Host 会话中的 PM 驱动本 Host 流程，不能据此推断它会启动其它 Harness。[15] CCTeam 则明确提供跨 Harness 会话委派；任务提示和文件/技能路径被传递，各 Harness 保持独立会话并按各自机制读取项目指令，不会自动复制完整主会话历史。[14] 其 Flow 文档还明确写出并行任务共享项目 working tree、没有 per-hire 隔离；写任务要先创建各自 worktree 再派发。dsh-alpha（约 3 stars）列出 Codex、DSH、Kimi、ZCode 等 Worker，适合作为协议桥设计参考，不作为首期稳定底座。[16] 上述项目详细比较与边界记录在 [多 Harness 参考项目调研](./多 Harness 参考项目调研.md)。

分类	组件	本版落地方式
复用开源组件	候选：CCTeam 会话/跨 Harness 派发；Omnigent worker、worktree 与评审机制	M0 用最小任务比较两者的 Codex/DSH/Kimi 启动、隔离、事件、权限及恢复；固定发行版本及源码提交后再定底座。
复用已有自研组件	SQLite 控制器、角色鉴权、MCP 网关、DeepSeek 代理、浏览器网关	逐项验证兼容性；保留审计、乐观锁、证据摘要和失败现场逻辑，调整固定阶段约束。
复用开源组件	Git worktree、bubblewrap、Playwright、Python MCP SDK	分别承担任务目录、操作系统边界、浏览器验证和工具协议。
需自研组件	SQLite 流程门禁、角色权限网关、任务卡与统一账本桥接、审计与角色凭据隔离；ZCode 适配（若候选框架不支持）	不重写 Agent 推理循环。按 M0 结果选择现成 DSH/Kimi 适配；ZCode `app-server` 是自有协议，需验证桥接或受限 headless 方案。
后续扩展项	Vibe Kanban 界面、更多 Harness、远程执行、自动角色选择	首期复用其任务隔离思路；不同时部署第二套任务平台。成本调度不纳入。

Omnigent 社区 Harness 插件接口面向 Direct / Headless 扩展，暂不支持任意第三方 Native TUI 插件。[4] Kimi Code 有 Omnigent 已有实现；DSH 有可供核验的 ACP profile；ZCode `app-server` 使用自有 ZCode Protocol，不能按 ACP 直接配置。若采用 bridge，必须证明工具请求经权限策略、可中断且结果可审计。

各 Harness 接入条件

Codex 优先验证 Omnigent 现有适配与官方 CLI 的订阅登录；目标模型仍为 GPT-6.1 Sol，实际可用性在原生 Codex 中确认。Kimi Code 的第三方 Base URL、模型配置和工具调用兼容性必须单独验证，不能据 Omnigent 的官方登录示例假定已经兼容第三方供应商。[5][6]

DSH 登记 CLI 来源、版本和无头参数后实现插件。ZCode 官方已有 CLI 源码；须核验本机 CLI、Linux 运行、机器输出和中断接口。[7]

本机预检（2026-10-12）：`dsh --profile acp --help` 和 `kimi acp --help` 已确认 ACP stdio 入口存在；尚未做 ACP 握手与真实模型请求。Omnigent CLI 有 Codex/Kimi/Polly 入口而无 DSH 内置项。隔离安装的 Codex CLI 状态为未登录。ZCode 0.16.9 有 `--prompt`、`--json`、`--disallowed-tools`、`--mode guarded`；其 `app-server` 是 ZCode Protocol，不是 ACP。以上仅为 CLI 接口预检，不计作 Harness 接入通过。

当前进度与迁移边界

2026 年 10 月 12 日盘点：WSL 用户 `mvpdev` 的 `/home/mvpdev/worktrees/multi-agent-workflow` 已有控制器与网关代码，包含未提交改动；任务清单列出真实项目闭环待验收。本次未重跑运行验收。

旧工作树、数据库和配置保留。先在新目录验证 Omnigent；通过后导入任务摘要并重新建立权限映射。每个任务仅有一个权威账本，旧任务标记为旧运行时管理，新任务进入新账本；禁止两个控制器同时推进同一任务。

3 任务流程与交接规则

流程根据任务选择。只读调查执行“问题与范围 → 调查 → 证据审阅”；小改动执行“简短任务卡 → 实现 → 测试 → 独立评审 → 交付”；跨模块或架构任务执行完整 spec → plan → build → test → review → ship。任务卡须说明采用路径及检查项不适用的理由。

已有明确授权的日常任务不重复询问每一阶段。范围变化、重大架构取舍、失败处置及外部发布仍遵守用户授权与项目规则。若用户明确说“按流水线处理”，必须执行全局 AGENTS.md 中 planner → planner_cross → 用户确认的指定流程；该触发规则不被本方案替代。

独立于 Agent 的任务账本

沿用现有 SQLite 服务作为本期任务权威记录，Omnigent 会话状态作为运行观测。账本状态为：待执行、执行中、待评审、受阻、待授权交付、完成、取消。阶段检查项作为证据字段，不再只允许六阶段机械相邻跳转。

Agent 提交产物与建议，控制服务按任务、授权与证据放行；revision、事务和 request_id 拒绝过期审批及重复提交。所有派发入口接到账本，内部子 Agent 继承任务权限与审计；未验证的旁路派发禁用。

代码变更须范围已授权、当前测试证据有效、独立评审通过且阻断问题清零。测试不适用须说明理由并经评审确认；本地提交按项目 AGENTS.md 执行。

推荐一次授权项目、任务分支前缀和草稿 PR 创建权限；此后通过测试与评审的任务由独立发布器自动 push 并建 Draft PR，无需每次重新确认。未取得该授权时先输出本地交付包。合并、部署、强推、删除分支不包含在草稿 PR 授权内；代码 Agent 始终不持有发布凭据。

交接契约

任务卡包含 task_id、父任务、项目、基线提交、目标、验收条件、依赖、允许路径、Harness / 角色 / 模型、credential_ref、状态及 revision。Worker 接收本任务上下文与产物引用。

回传结果包含完成说明、变更文件、diff 摘要、测试日志引用、风险、未完成事项和退出状态。测试与评审绑定基线提交、当前 diff 摘要和验收条件摘要；代码或验收条件变化后，旧证据失效。结构化结果不完整时进入受阻，不根据自然语言“已完成”放行。

并行与失败处置

首轮最多同时运行 2 个 Worker。每个独立写任务使用不同 worktree；同一 worktree 同时只能有一个写者。依赖未完成的任务排队，跨分支集成由受控集成器执行，集成后的完整改动重新测试和评审。

测试、评审或工具失败时保留 worktree、浏览器证据和日志，形成修复简报；用户确认后才进入修复轮，不自动修复、不自动回滚。涉及范围外问题时遵守项目适用的子流水线规则。恢复任务先检查进程、基线和产物，不能直接重放上次写命令。

4 权限与云端调用治理

Codex 主控负责提出调度决策；控制服务负责验证能否执行。角色身份从启动配置和独立凭据绑定，不能由 Agent 在工具参数中自报。审查业务代码只读；运行时可以写各自临时目录与受控会话元数据。

角色	允许操作	强制限制
主控与审查	读源码、diff、证据；主控可请求建任务和派发 Worker	不写业务代码，不获得人工审批凭据，不执行任意 Shell。
DSH 与 ZCode 写者	写批准路径，申请固定构建 / 测试命令，读 Git 状态和 diff	禁止 push、生产数据库写入、修改网关配置、访问其他任务凭据。
测试器	执行批准的检查与浏览器断言	测试使用隔离临时输出；不得借测试命令改源码、访问生产系统。
集成与发布器	校验产物、生成本地提交；通过门禁后自动创建已授权的 Draft PR	发布凭据不提供给 Agent；固定仓库与分支前缀，禁止强推、自动合并，证据过期即停止。

默认免逐工具审批：读文件、批准路径内写入、固定构建和测试自动执行。首次启用时验证所选 bypass 模式仍受外层策略、操作系统边界和审计控制；权限变更、任务外操作和破坏性操作由网关拒绝或请求明确授权，避免把常规开发变成逐次点击确认。

MCP 网关只能控制经它调用的工具。原生文件、Shell、Git 必须通过各 Harness 的执行前权限钩子与操作系统隔离共同约束；不能以 MCP 工具列表证明原生工具安全。命令使用固定参数接口；目录需校验真实路径、符号链接、允许范围和 Git 元数据访问。

Omnigent 文档明确 MCP Server 子进程不自动处于其 OS Sandbox 内，因此 MCP 服务自身也须限权和隔离。逐 Harness 核验原生工具拦截和审计覆盖；做不到的 Harness 暂不开放写权限。Ubuntu 24.04 的 namespace / AppArmor 限制须实测，沙箱失败不得自动降级为无沙箱。[8]

认证与模型

Codex 通过官方 ChatGPT 登录保留订阅认证，不把 OAuth 凭据改造成通用第三方 API Key。清除可能覆盖订阅选择的默认 Provider，并核对实际登录类型；额度不足或模型不可用时阻断，不转成付费 API 或其他模型。[5][6]

DSH、Kimi Code、ZCode 使用第三方云端接口，无自托管模型或专属节点。沿用已确认的 DeepSeek 请求 ID `deepseek-v4-pro-0813`，接受供应商服务端实际路由；`response.model` 只标注“供应商返回标识”，不能证明底层权重版本。其他角色的模型 ID 在 M0 登记后固定。

各 API 角色分别使用供应商子 Key 和独立配置，禁止全局环境共享。Codex 同一官方账号下的会话使用角色隔离和最小权限，不伪称为独立订阅账号。供应商不支持角色子 Key 或 CLI 不能隔离配置时，记录缺口供用户取舍。首期不实施敏感代码识别或禁止外发规则；任务所需代码按既定云端调用方式处理。

审计与重试

每次调用记录 task_id、角色、Harness 及版本、request_id、工具名或请求模型 ID、返回模型标识、token 用量、耗时、状态码、结果及错误类别。不可获得的字段写 null 并说明来源，不推测。审计保存参数摘要与产物引用，不记录完整提示、密钥、代码原文或原始 HTTP 包；会话正文与审计分开限权，关闭正文调试日志。

每个任务执行轮沿用最多 2 次额外失败重试预算。超时、429 和可恢复服务错误统一分类并按 Retry-After / 退避处理；401 / 403 不重试。核验 CLI、SDK、代理的内部重试，避免叠加。结果不明的写操作和整个 Agent 任务不自动重放；耗尽预算即受阻，不自动换模型。审计落盘失败时拒绝写操作。

5 Git 浏览器与 Windows 落地规则

真实项目与 Git

平台运行于 WSL2 Ubuntu 24.04。试点基线为 `/home/mvpdev/projects/mingjing-prototype-base`，任务副本为 `/home/mvpdev/mvp-worktrees/mingjing-prototype-onboard`。Windows 源目录属于 `D:\mingjing-develop` 父仓库。

新任务登记源目录、源提交或文件摘要、WSL 基线与任务分支；仓库和依赖使用 WSL Linux 文件系统。worktree 并非权限沙箱，共享 Git 元数据必须由网关限制；Worker 不得获得整个父仓库及其其他产品目录的读写权限。

失败仅冻结本任务。禁止自动 `git checkout -- .`、`reset --hard`、`clean -fd` 和全盘重置。人工确认处置范围后才允许清理任务文件或归档 worktree。补丁使用 Git 原生文件输出并保留二进制、重命名和删除信息，应用前执行检查；不经 PowerShell 文本管道转码。

WSL 快照不自动同步回 Windows。回写前比较源文件基线摘要，源端已改动则停止并列出冲突；仅处理批准的任务文件，包括新增、删除和重命名。合并后在最终目标目录重新执行适用检查，再按项目规则本地提交。

浏览器验收

每个任务使用独立临时 user-data-dir 或隔离 context；不连接个人 Profile。Profile 独占锁绑定任务和进程，启动前判断残留进程，不能直接删除正在使用的锁。

使用明确的本地站点 URL，导航后等待目标 DOM / 业务就绪信号；出现 about:blank、重定向异常、页面错误或超时时保留证据并阻断。记录最终 URL、关键 DOM 断言、控制台及页面错误、网络失败和 HTTP 错误；截图用于辅助检查。允许的已知噪声须逐条记录理由，不忽略全部控制台错误。失败保留 Profile 与证据，成功后按任务归档规则清理。

Windows 与 WSL 高频问题

PowerShell 文件写入固定 UTF-8，WSL 使用 UTF-8 locale；中文路径通过参数传递，避免拼接 Shell。项目统一换行策略，核验大小写、可执行位和符号链接，Windows 与 WSL 不同时修改同一任务目录。

代理、OAuth 回调和本地端口分别检查 Windows 与 WSL 可达性；localhost 代理警告不等于 Ubuntu 初始化失败。按实际网络模式配置代理，不盲目关闭隔离。停止或恢复 WSL 前记录活动任务；重启后核对进程与端口归属，不能重复启动写任务或批量杀死浏览器。

运行平台绑定回环地址，固定 Python、Node、Harness、MCP 与浏览器版本。第三方依赖安装由受控环境步骤完成；任务源码、Skills 和项目插件不能自行扩大权限或加载新的可执行配置。

6 实施里程碑与 M0 核验

各里程碑交付配置、证据和通过 / 失败结果；核心兼容冲突先形成最小复现及备选对比，再请用户取舍。

M0 验证底座与四个接入路径

1. 固定 Omnigent 发行版本、提交和许可证；安装于新目录，登记现有工作树及数据库备份位置，避免覆盖未提交资产。

2. 核验 WSL 的 bwrap、namespace、进程中断与端口；分别验证 Runner 和 MCP Server 的权限边界；确认开启免逐工具提示后，越权仍被拒绝。

3. 四个 Harness 均登记 CLI 来源、版本、启动方式、认证类型、请求模型 ID、返回标识与输出协议。Codex 在官方 CLI 中验证 GPT-6.1 Sol 和订阅认证。

4. 对三条第三方路径分别验证 Base URL、TLS、鉴权、文本响应、流式输出、多轮上下文、工具调用和错误返回；已有 ZCode 或 OpenCode 成功记录不能替代另一个 CLI 的核验。

5. 用受控模拟验证超时、429、401 / 403 及重试预算；真实接口做基础连通性与耗时检查，不以压垮供应商触发限流为测试手段。

6. 各 Harness 执行一次只读文件调查，核对真实 CLI 进程、工具事件、结果引用和审计；测试停止后无残留写进程。

7. DSH、ZCode 完成最小 Headless 插件验证；不能取得机器输出、执行前控制或完整工具事件时，登记阻断，不通过 UI 自动点击冒充正式适配。

M0 输出 `compatibility-matrix`：逐 Harness 标记通过、只读可用、待适配或阻断，并附证据。仅模型 HTTP 200 不算接入通过。

M1 建立第一条真实协作链

Codex 派发给已通过 M0 的 Kimi Code 做只读调查，结构化回传并由 Codex 汇总；随后接入 DSH 完成一个隔离小改动，经确定性测试器与只读评审交付。连接现有账本和网关，验证故障阻断及恢复；有发布授权时核验自动 Draft PR，无授权时验证本地交付包。此时只证明部分链路，不能宣称四 Harness 完成。

M2 接通四个 Harness

加入 ZCode 前端写任务，完成 Kimi 独立评审和 Codex 终审；跑通 DSH、ZCode 不同 worktree 的协作与最终集成验证。每个 Harness 均须收到真实任务并产生产物，满足角色权限、凭据隔离和完整工具审计。

M3 在真实项目完成质量验收

在原型的 WSL 副本执行 10 个小中型任务，覆盖只读调查、前端交互、逻辑修改、跨文件任务及失败恢复。至少 1 个任务经用户确认完成回写检查，验证 WSL 与 Windows 的交付路径。系统代码、配置、锁文件、适配器源码、操作手册、审计样例和未解决问题清单一并交付。

M0 不通过时的取舍

若 Omnigent 的核心隔离或原生驱动不满足要求，提交“Codex 官方 app-server / CLI + 已有网关 + 自定义适配器”备选及开发量，由用户取舍。[9] 单个 Harness 不兼容第三方 API 时，明确列出更换接入或推迟的影响，不能用模型替身算接入成功。

7 验收标准与主要风险

指标	首期通过标准
四 Harness 实接	四者都有真实 CLI / 原生运行记录、独立会话、任务输入、工具事件和可核对产物；双 Agent 演示不计完整交付。
自动化体验	已批准范围内常规工具无需逐次确认；只有通过测试与评审的产物才能自动创建已授权 Draft PR，未授权仓库或分支被拒绝。
交付成功率	10 个预先登记任务至少 9 个通过全部适用验收；失败和放弃均计入分母，人工补写代码的任务单列辅助成功。
人工干预	已按任务预定的授权单列；成功任务非计划干预平均 ≤2 次，同时报告总次数及人工补写次数。
缺陷逃逸	交付后 7 日内高严重度缺陷为 0；报告逃逸数与全部确认缺陷数，逃逸比例目标 ≤10%。无确认缺陷时记 N/A。
权限与门禁	原生工具及 MCP 越权拒绝检查全部通过；评审写业务代码、Worker push、任务外写入和无授权数据库写入均为 0。
证据与审计	每个工具调用均有脱敏审计；缺失模型字段如实标注。过期 diff / 过期审批不能放行，不产生密钥或代码原文审计。
恢复与集成	重启、取消、超时后无重复写入；失败现场可检查；并行任务不串目录，集成后重新验证。
浏览器与回写	使用任务独立 Profile，URL / DOM / 错误检查完整；Windows 源端冲突可阻断，批准范围外文件无变化。

10 个任务为试点，稳定使用前累计观察至少 30 个。token 与耗时仅用于诊断和审计。

高风险：Omnigent alpha 与适配变动；DSH 官方标注 developer preview 且可能有破坏性兼容变更。固定版本并先在独立环境验证/升级；DSH / ZCode 适配与 Kimi 第三方 Provider 配置经 M0 实测后才开放写权限。[17]

高风险：原生工具绕过、Git 元数据与 MCP 沙箱缺口，逐项做越权验证。第三方供应商可能共用，须记录共同故障和模型映射风险；跨 Harness 评审不等于跨模型或跨供应商评审。

中风险：旧状态迁移和 Windows 回写，保留旧资产、绑定证据、逐任务迁移并检查冲突。复用与稳定性实质冲突时由用户取舍。

8 官方依据

以下资料用于核对组件能力；本方案的职责、门禁和验收数值为本项目设计。

[1] Databricks Omnigent 发布介绍与 alpha 定位

[2] Omnigent 支持矩阵与执行模式

[3] Polly 协作机制与默认权限行为

[4] Omnigent 社区 Harness 插件边界

[5] Omnigent 各 Harness 认证和配置

[6] OpenAI Codex 官方认证文档

[7] Z.ai ZCode 官方仓库与 CLI

[8] Omnigent OS Sandbox 覆盖与限制

[9] Codex 官方 app-server 接口

[10] AWS Labs CLI Agent Orchestrator 与本次 GitHub stars
[11] CAO 多 Provider 委派与 profile 配置
[12] Vibe Kanban 与本次 GitHub stars
[13] Vibe Kanban 支持的 Coding Agent
[14] CCTeam 跨 Harness 会话派发与本次 GitHub stars
[15] Morning Star 跨 Host 安装与本次 GitHub stars
[16] dsh-alpha 的 Worker/运行时清单与当前限制
[17] DeepSeek Harness 官方仓库中的 Developer Preview 与兼容性说明